DEDEKIND: An Excursion in Structuralist Mereology in C++23

Vincent Kräutler

March 30, 2026

Abstract

This paper presents `dedekind`, an experimental C++23 library that reifies mathematical structures as transparent, type-level definitions. Unlike traditional object-oriented approaches, `dedekind` utilizes modules and concepts to transform the C++ compiler into a constraint satisfaction engine. We specifically demonstrate how this approach enables the formal construction of the continuum $\mathbb{R}$ via the Dedekind cut, allowing for the exact, compile-time resolution of real and transcendental numbers. By hoisting these formal invariants, we eliminate abstraction overhead, providing a high-performance alternative to Python's symbolic systems while offering a level of axiomatic rigor that complements the safety-first philosophy of languages like Rust. We illustrate this by providing a unified syntax that allows formal set-builder notation—such as $\{x \in \mathbb{R} \mid \mathbf{P}(x)\}$—to be expressed and resolved directly within the host language's type system, bridging the gap between formal specification and high-performance execution.

1 Introduction

A powerful abstraction in mathematical logic is set-builder notation: the ability to define a collection of objects $\{x \in \mathbb{R} \mid \mathbf{P}(x)\}$ based purely on the selection of a universe (in this case $\mathbb{R}$) and their satisfying a predicate $\mathbf{P}(x)$. In traditional systems programming, however, this declarative elegance is lost. Developers are usually forced to manually translate these logical intentions into imperative loops, filter functions, and temporary buffers. This translation is not only prone to error but also opaque to the compiler, which cannot optimize a "black-box" loop using the formal properties of the set it is iterating over.

In this paper, we propose `dedekind`, a domain-specific language (DSL) for C++23 [1] that embeds set-builder notation directly into the host language syntax. Recently added C++23 features—such as modules, `concept`, `consteval`, and non-type template parameters (NTTP)—allow developers to define complex mathematical species using a syntax that mirrors formal logic. Crucially, these are not mere runtime "filter" objects; they are compile-time structural specifications [2]. Just as a C++ concept defines a set of behaviors without mandating a specific memory layout, Structuralism [3] treats mathematical entities as positions in a system of relations.

The aim of the `dedekind` library is to serve as a high-performance bridge for the computational sciences, specifically aiming to narrow the gap between C++ and the symbolic algebra systems prevalent in the Python ecosystem. While Python allows for the expressive definition of the Continuum, the execution is often decoupled from the hardware's potential. Conversely, while the Rust programming language provides a "safety-first" approach to systems programming, its trait system currently lacks the "white-box" structural transparency required to hoist complex mereological invariants, such as the Dedekind cut, into the heart of the optimization pipeline.

By reifying the Dedekind cut—defining real numbers $\mathbb{R}$ as partitions of the rationals $\mathbb{Q}$—directly within the C++ type system, we enable the exact representation of transcendental

numbers like e and π at compile-time. This methodology allows the C++ compiler to act as a physical guardrail for mathematical truth, ensuring that a "Ring" or a "Field" is not merely a label, but a verified invariant.

However, reifying a formal ontology in a statically-typed **systems** language presents significant challenges compared to "pure" functional languages like Haskell [4] or Agda [5], which have native support for higher-kinded types, dependent types and first-class functions. Challenges encountered include:

- **Template Template Parameters**: C++ is notoriously bad at true Category Theory because it lacks higher-kinded Types (HKTs). As a result `Functor<F>` or `Monad<F>` are defined in terms of `template template`, which have significant edge cases (e.g., they don't play well with aliases or multiple template arguments).

- **The Lambda Hurdle (Opaqueness)**: Unlike the first-class, inspectable functions in the ML family, C++ lambdas are unique, anonymous types. They do not inherently expose their domain or codomain to the type system. A morphism's domain and codomain must be inferred at the call site or explicitly provided by the programmer. Although if used as *Non-Type Template Parameters* (NTTP), they can be "inspected" them via `concept` checks or by wrapping them in a type that carries its own signature metadata.

- **Decidability vs. Instantiation**: While dependent-type systems use proof-search, C++ relies on template instantiation. This introduces the risk of "Late Detection," where a mathematical violation is only discovered when a set is materialized, rather than when it is defined. This can be mitigated by forcing evaluations into `consteval` and `constexpr` contexts.

- **The Performance-Rigidity Tradeoff**: High-level abstractions in C++ often incur a "template tax" in compile times. We utilize C++23 **Module Partitions** to create a directed acyclic graph of truth, caching structural invariants to ensure that formal rigor does not collapse the build pipeline.

- **The Specialization Constraint**: Unlike the unified pattern matching of functional languages, C++ relies on Partial Template Specialization to resolve structural properties. This creates a "Rigidity Challenge": the compiler cannot readily deduce that a property of A also applies to $A \cup B$ without explicit boilerplate. We address this by using `concept ... requires` as logical compile-time predicate, thereby simulating the polymorphic reach of a true dependent-type system.

- **The Precedence-Associativity Constraint**: A significant friction point in embedding categorical notation within the C++ grammar is the language's fixed set of `operator` candidates and fixed precedence and associativity rules. While the `operator»=` is used to model the Monadic Bind ($\gg=$), the ISO C++ standard defines compound assignment operators (including `»=`, `«=`, `+=`, `*=`, etc.) with **right-to-left associativity**. Consequently, a standard monadic chain such as $m \gg= f \gg= g = (m \gg= f) \gg= g$ is parsed by the compiler as `m »= (f »= g)`. As this makes no sense in a monadic chain, explicit grouping, e.g., `(m »= f) »= g` is required. As an alternative, we provide `operator»` ("Highway Notation"), but with slightly different categorical semantics.

We note that while these kinds of restrictions may make C++ appear frugal when compared to a "proper" functional programming environment, the small runtime footprint of C++ will still make the approach worthwhile for many use cases. Here, we also note that the Rust programming language may have a similar thrust as a strongly type-checked systems programming language. However, the aim of the DEDEKIND library is more focussed on serving the high-performance Python ecosystem. While Python's symbolic algebra systems, such as SymPy, offer unparalleled

declarative ease, they often act as "slow" oracles that require manual translation into C++ for production performance. DEDEKIND aims to narrow this gap by allowing the C++ compiler itself to understand the symbolic logic of the Continuum.

By implementing the Dedekind cut—defining real numbers $\mathbb{R}$ as partitions of the rationals $\mathbb{Q}$—directly within the type system, we enable the exact representation of transcendental numbers like e and π without floating-point approximation. This puts dedekind in a unique position relative to Rust: while Rust provides superior memory safety through its borrow checker, C++23's advanced template metaprogramming and modules allow for a deeper, "white-box" formal verification of mathematical structures that Rust's current trait system cannot easily replicate.

Despite these constraints, the contributions of this work include:

- **Intensional-to-Extensional Materialization**: A strategy for partial, lazy evaluation, where a set defined by a symbolic rule is only converted into machine instructions (extensional data) when an operation requires it. This enables the evaluation of Infinite Sets (e.g., the Naturals $\mathbb{N}$) within a finite execution environment.
- **Dedekind Cuts**: exact representation of both finite as well as infinite series, enabling the Dedekind cut to define the real numbers $\mathbb{R}$ (including the transcendental numbers $e, \pi, \sqrt{2}$) *exactly*.
- **Complex and Dual Numbers**: *exact* representation of differentiable functions through augmented real number lines. Perhaps we can even prove Euler's formula, but I am not certain.
- **Mereological Pruning of Predicates**: Because the library understands the logical structure of the set-builder predicates, the compiler can prune the search space of the set. For example, a set defined by a contradiction $\{x \in \mathbb{S} \mid \mathbf{P}(x) \wedge \neg P(x)\}$ is collapsed into the empty set $\emptyset$ at compile-time, emitting zero machine instructions.
- **Set-Builder Isomorphism**: A syntax-driven mapping that allows expressions like

$$\mathrm{Set}\{\mathrm{x} \,\%\, \mathbb{R} \wedge (\mathrm{x} + 5 < 10)\} \tag{1}$$

  to be resolved at compile-time. We demonstrate how the C++ type system acts as a Constraint Satisfaction Engine to verify the validity of these sets.
- **Zero-Overhead Semantic Mapping**: We show that these high-level abstractions result in machine code that is identical to hand-optimized assembly, effectively proving that formal rigor does not necessitate a "performance tax."

1.1 A Structuralist Type Theory: C++ vs. Nominal Systems

1.1.1 Nominal vs. Structural Ontology

In traditional object-oriented paradigms (e.g., Java, C#), the ontology is *nominal*. A type T belongs to a category $\mathcal{C}$ only if it explicitly declares its lineage via inheritance (e.g., `class T implements Monoid`). In the Dedekind framework, we reject this biological taxonomy in favour of a **Structuralist Ontology**. Using C++20 Concepts, we treat types as mathematical objects defined entirely by their available morphisms. A type T *is* a `SmallCategory` if and only if it possesses the required algebraic structure $(\otimes, e)$; its name and declaration are irrelevant.

1.1.2 The Idiosyncratic Fit: C++ as a Formal Verifier

While functional languages like Haskell are often viewed as the natural home for Category Theory, C++ offers an *idiosyncratic* but rigorous fit for structuralism. By utilizing `static_assert`

against structural concepts, we transform the compiler into a formal proof assistant. This allows for a "proven by construction" methodology where the structural integrity of the mathematical topos is verified before a single byte of machine code is emitted.

1.1.3 Mapping ETCS to C++ Structural Primitives

The following table maps the fundamental axioms of the *Elementary Theory of the Category of Sets* (ETCS) to their structural implementations within the Dedekind library:

Table 1: ETCS Axioms and C++ Structural Equivalents

ETCS Axiom	**C++ Language Feature**	**Dedekind Concept**
Composition	`std::invoke` / `operator»`	`IsSemigroupoid<T, Op>`
Identity	`identity_trait<T, Op>`	`IsSmallCategory<T, Op>`
Functionality	`std::is_invocable_v<F, A>`	`IsArrow<F, A, B>`
Terminal Object	`s(x) -> Logic::top`	`IsTerminalObject<T>`

This mapping ensures that our software implementation is not merely a *simulation* of set theory, but a direct *instantiation* of its algebraic laws.

The structural mereology [6] approach in `dedekind` defines sets via formal relations—union ($X \cup Y$), intersection ($X \cap Y$), and parthood ($X \subseteq Y$) and last but not least element ($x \in Y$) – prior to implementation. This mirrors structuralist foundations such as Lawvere's Elementary Theory of the Category of Sets (ETCS) [7] and Cartesian Closed Categories (CCCs) [8, 9]. By prioritizing *structural transparency*, this methodology fundamentally departs from object-oriented encapsulation. In a sense, it encodes the well known rule of thumb of "composition over inheritance" [10]. In a standard `std::set`, the internal representation is an opaque implementation detail, invisible to the type system's logic. Conversely, in `dedekind`, the *type is the definition*. Because the set's constituents—predicates, universes, and mereological relations—are hoisted into the type signature as first-class members, the C++ type system and the `CMake` toolchain function as a Constraint Satisfaction Engine (CSE). This allows the compiler to inspect, decompose, and optimize a set based on its logical essence rather than its storage layout, trading the 'black-box' safety of encapsulation for the 'white-box' provability of formal mereology.

```
#include <concepts>
#include <type_traits>

// Verifies the "Element Of" relation (x in Y)
template <typename T, typename S>
concept IsElementOf = requires(T x, S s) {
    { s.contains(x) } -> std::same_as<bool>;
};

// Verifies "Parthood" or "Subset" relation (X subset of Y)
template <typename Sub, typename Super>
concept IsSubsetOf = requires {
    requires std::derived_from<Sub, Super> ||
             requires { typename Sub::is_subset_of<Super>; };
};

// CSE Pruning: Collapsing contradictory intersections to EmptySet
template <typename SetA, typename SetB>
struct Intersection {
    using type = std::conditional_t<
        HasContradictoryPredicates_v<SetA, SetB>, // Evaluated at compile-time
```

```
        EmptySet,
        InternalIntersection<SetA, SetB>
    >;
};
```

Listing 1: Compile-time mereological verification using C++23 concepts.

By reifying a subset of the set-builder notation, dedekind turns the C++ translation unit into a symbolic laboratory. The programmer is free to reason in the language of Cantor, while the compiler produces the performance of a naked C++ loop.

2 Implementation

The methodology of `dedekind` is predicated on the translation of formal mathematical structures into a stratified registry of C++23 modules. This stratification allows for a "bottom-up" synthesis of complex mathematical objects, where each level serves as a verified foundation for the next. The intention being that the flow of compilation would mirror the order in which objects might get introduced in a textbook or a lecture. I.e. objects which are compiled first would tend to be more general, more foundational (either in the mathematical sense or in terms of embedding in the C++ host language) whereas objects which get compiled later will be more elaborate and specialized but also perhaps more practical to the user.

To ensure the decidability of the ontology and mitigate the 'template tax,' we utilize a strictly enforced build graph (see Figure 1). This DAG mirrors the formal structure of a mathematical proof: *Axiom* $\rightarrow$ *Theorem* $\rightarrow$ *Proof*. By isolating foundational mereological axioms into the `:mereology` partition, the compiler produces a Binary Module Interface (BMI) that acts as a 'cached truth.' This physical stratification prohibits circular reasoning at the compiler level; a partition can only reference previously verified structural properties. Subsequent partitions, such as `:algebra`, import these BMIs as established lemmas, allowing the compiler to focus solely on the synthesis of higher-order laws without the risk of recursive definition or re-instantiation of the underlying species. In this architecture, the C++ build system serves as a physical guardrail for the integrity of the formal derivation.

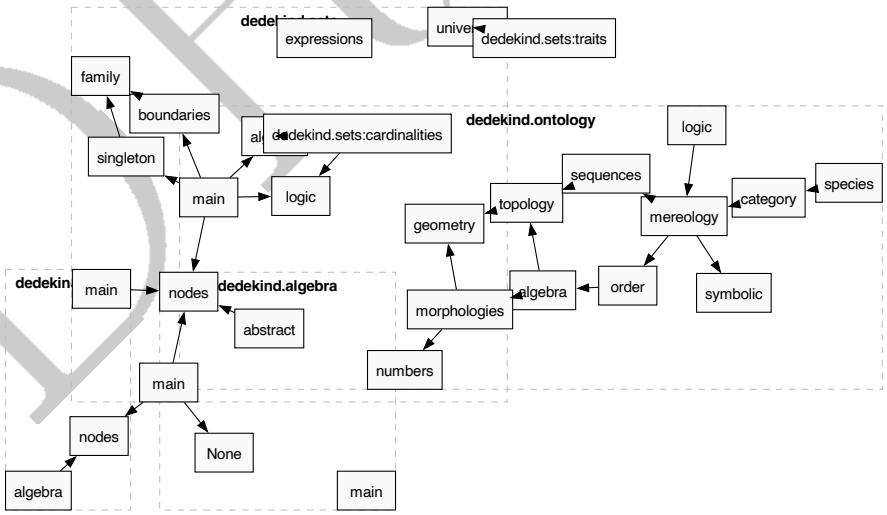

Figure 1: The Ontological Build Graph: Automated dependency resolution.

2.1 The Unified Dedekind Ontology

The `dedekind.ontology` module serves as a central registry of structural truths. It aggregates skeletal, mereological, and algebraic laws to enable the synthesis of the *Continuum* from the *Discrete*. By adopting a *Structuralist* posture—viewing mathematical objects as positions within a structure rather than isolated collections—we facilitate zero-overhead formal verification within the C++ type system.

The stratification of the `dedekind.ontology` into C++ module partitions mirrors the constructive dependencies of formal mathematics. At the foundation, an embedding of Category Theory (Level 0) within the type system provides the primary substrate; as it makes the fewest assumptions—treating objects merely as nodes in a graph of morphisms—it establishes the "highways" (Functors and Kleisli Triples) necessary for all subsequent synthesis. Because higher algebraic structures—such as Groups, Rings, and Fields—are formally defined by the operations performed upon sets, the build order mandates that Level 1 (Space and Mereology) precedes Level 3 (Algebra). This ensures that the "Soul" of the system (its laws of action and harmony) is always grounded in the "Body" (the structural presence of its parts). By the time the compiler reaches the `:algebra` partition, the mereological validity of the underlying species is already a type-checked invariant within the translation unit. This hierarchy transforms the C++ module mapper into a directed acyclic graph of mathematical truth, where the linker cannot even begin its work until the ontological prerequisites are satisfied. The strict directed acyclic graph (DAG) of the dedekind module partitions mirrors the Bourbaki style of 'Mother Structures' [10], where the physical order of compilation serves as a proof of logical consistency.

Level -1 to 0: Foundations (Bricks and Cement) At the base lies `:logic`, defining the subobject classifier Ω and predicate logic. This is supported by `:species`, which provides reified machine primitives, and `:category`, which establishes the "Highways" of the system via Morphisms, Functors, and Kleisli Triples.

Level 1: Space (Body and Relation) The `:mereology` partition defines the rules of presence (parts and wholes), while `:order` and `:topology` establish the rules of relation and closeness (posets, lattices, and open sets).

Level 2: Scale (Measurement and Path) The `:sequences` module maps magnitudes to enumerations, providing the necessary machinery for limits and convergence.

Level 3: Soul (Harmony and Action) This level implements the pure algebraic laws (`:algebra`) including Groups, Rings, and Fields, alongside their realized `:morphologies` (Cyclic, Ordered, and Archimedean forms).

Level 4: Realization (The Registry) The final level, `:geometry` and `:numbers`, realizes the continuum through the species registry, formally constructing the inclusion $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R}$.

The physical build order of the `dedekind.ontology` module mirrors this conceptual hierarchy. By enforcing a linear dependency graph—where `:mereology` is compiled before `:algebra`—we ensure that the compiler's proof-search for a Ring's identity element is grounded in the prior verification of its mereological subobjects. This hierarchy transforms the C++ module mapper into a directed acyclic graph of mathematical truth.

2.2 The Compiler as a Deterministic Proof Assistant

The core of `dedekind` is a recursive template evaluator that treats C++ `concepts` as logical predicates. Unlike traditional template metaprogramming that relies on SFINAE, we utilize C++23's `requires` expressions to perform "structural inspection" of types.

In the `dedekind` framework, the C++ compiler acts as a decidability engine for our mathematical ontology. Rather than relying on traditional class hierarchies, we use C++23 Concepts to define the structural requirements of a species. A type is not explicitly inherited from a base; instead, it is verified to be compliant with a species' `concept` through its intrinsic properties.

This verification is supported by Template Specialization to define axiomatic base cases and SFINAE to prune invalid overloads. We leverage the Curiously Recurring Template Pattern (CRTP) within our internal wrappers to provide a unified interface for disparate types without the overhead of dynamic dispatch. By conducting these checks in a `constexpr` context and utilizing Move Semantics for resource management, we ensure that the "Proof" of a mathematical construction is resolved entirely at compile-time.

```
// Verifying that the intersection of disjoint sets is empty
using Evens = dedekind::set<int, [](auto x){ return x % 2 == 0; }>;
using Odds  = dedekind::set<int, [](auto x){ return x % 2 != 0; }>;

using Intersection = dedekind::meet_t<Evens, Odds>;

// The compiler resolves this to the Empty Set species
static_assert(dedekind::is_empty_v<Intersection>);
static_assert(sizeof(Intersection) == 1); // No machine state required
```

Listing 2: LCompile-time inference of set invariants

2.3 The Semantic Mapping

Crucially, the `dedekind.ontology` acts as the definitive mapping between formal mathematical nomenclature and the concrete syntax of the C++23 language. Rather than introducing a foreign vocabulary, we anchor established axioms directly into the standard operator set and primitive types. For instance, the mereological *Sum* and *Product* are reified as the bitwise operators `|` and `&`, while the *Mereological Complement* (the remainder) is bound to the negation operator `!`.

By establishing this explicit correspondence—where `operator<=` serves as the formal proof of *Parthood* ($\sqsubseteq$) and `operator/` represents the *Quotienting* of a species—we ensure that the resulting code is not merely a simulation of a mathematical model, but a direct execution of it. This semantic alignment allows the programmer to write expressions that are readable as equations, while the compiler treats them as a sequence of verifiable type-transformations. In this architecture, the C++ `concept` becomes the "Gatekeeper of Truth," ensuring that a species cannot participate in an operation (such as a Join-Semilattice union) unless it has first provided a manual or structural certification of its algebraic invariants.

2.4 The Registry of Structural Proofs

The transition from a raw computational graph to a simplified mathematical identity requires a formal "Seal of Truth." In `dedekind`, we implement a *Traits Registry* that forces the developer to explicitly certify the equational axioms of a species before it can participate in the lattice-based optimizations of the dispatcher.

This is achieved through a dual-layered system of **Trait Ledgers** and **Discovery Concepts**. For each fundamental algebraic law, we define a primary template (the ledger) which defaults to `false`:

```
export template <typename T, typename Op>
struct is_associative : std::false_type {};

export template <typename T, typename Op>
inline constexpr bool is_associative_v = is_associative<T, Op>::value;
```

Listing 3: The Associativity Ledger in :species

Table 2: The Dedekind Semantic Mapping: A Registry of Formal Correspondences

Mathematical Concept	**Symbol**	**C++ Operator / Type**	**C++ Concept**	**Basis**
Category Theory				
Kleisli Extension	$(T, \eta, \gg=)$	`operator»=`	`IsKleisliExtension`	`IsAssociative`
Kleisli Extension	$\gg=$	`operator»=`	`IsKleisliExtension`	`IsAssociative`
Kleisli Composition	$\circ\_K$	`operator»`	`IsMonad`	`IsAssociative`
Co-Kleisli Extension	$\gg=_{co}$	`operator«=`	`IsComonad`	`IsAssociative`
Morphic Injection (Unit)	η	`into<F>` / `singleton`	`IsMonad`	$\eta : T \to F(T)$
Morphic Extraction (Counit)	ε	`extract<F>`	`IsComonad`	$\varepsilon : F(T) \to T$
Characteristic Morphism	χ	`operator()`	`IsCharacteristic`	
Co-Kleisli Composition	$\circ\_CoK$	`operator«`	`IsComonad`	
Counit (Extraction)	ε	`extract<F>`	`IsComonad`	
Mereology				
Proper Parthood	χ	`operator()`	`IsProperPart`	`IsCharacteristic`
Parthood (Pre-order)	$\sqsubseteq$	`operator<=`	`IsPartOf`	
Mereological Sum (Join)	$\sqcup$	`operator|`	`IsJoinSemilattice`	
Mereological Product (Meet)	$\sqcap$	`operator&`	`IsMeetSemilattice`	
Universal Complement	$\neg$	`operator!`	`IsComplemented`	
Initial Object (Empty)	$\emptyset$	`∅<T>`	`IsInitialObject`	
Terminal Object (Universal)	Ω	`Ω<T>`	`IsTerminalObject`	
Set Theory				
Membership	$\in$	`operator()`	`IsSet`	`IsProperPart`
Subset	$\subseteq$	`operator<=`	`IsSubset`	`IsPartOf`
Countable Infinity	$\aleph_0$	`ℵ_0`	`IsCardinality`	
Continuum Cardinality	$\beth_1$	`ℶ_1`	`IsCardinality`	
Order Theory				
Pre-order	$\sqsubseteq$	`operator<=`	`IsPartiallyOrdered`	`IsPartOf`
Algebra				
Structural Origin (Zero)	0	`T::origin()`	`IsPointed`	
Additive Composition	$+$	`operator+`	`IsGroup`	
Additive Inverse	$-$	`operator-`	`IsGroup`	
Multiplicative Composition	$\times$	`operator*`	`IsRing`	
Multiplicative Inverse	$/, {}^{-1}$	`operator/`	`IsField`	
Associativity	$(a \circ b) \circ c$	`is_associative_v`	`IsSemigroupoid`	
Idempotency	$a \circ a = a$	`is_idempotent_v`	`IsIdempotent`	
Magnitude / Count	$\|S\|$	`s.size()` / `s.cardinality()`	`IsCardinality`	
Supremum / Infimum	$\sup, \inf$	`s.supremum(), s.infimum()`	`HasExtrema`	

A species S is only recognized as a `IsSemigroupoid` if it (or the ontology) provides a specialization of this ledger. To allow for "Interoperable Discovery," we provide a fallback mechanism that probes the species for internal certification:

```
export template <typename T, typename Op>
concept IsAssociative = IsMagmoid<T, Op> && requires {
    requires is_associative_v<T, Op>;
};

// Discovery Fallback: Probing the Species for an Intrinsic Proof
template <typename T, typename Op>
  requires requires { T::is_associative_for<Op>; }
struct is_associative<T, Op> : T::template is_associative_for<Op> {};
```

Listing 4: The Concept of Formal Association

By anchoring concepts like `IsJoinSemilattice` in these requirements (requiring both `IsAssociative` and `IsIdempotent`), we ensure that the *Lattice Dispatcher* never performs a "speculative" pruning. If the developer has not provided the proof, the dispatcher falls back to a safe, unoptimized `UnionSet` synthesis. This "Axiom of Responsibility" ensures that the performance of the "Naked Loop" is always mathematically justified by a prior formal certification.

2.5 Ontological Model Validation

To verify the integrity of the proposed mapping between formal axioms and C++ concepts, we employ a "Reverse Curry-Howard" validation strategy. Rather than simply assuming the correctness of our ontological translations, we use the inherent properties of C++ built-in types as an empirical proof of the reverse-engineered axioms.

By executing a suite of `static_assert` evaluations on primitive types—such as verifying that `bool` satisfies the requirements of an `IsJoinSemilattice` or that bitwise operations on `uint32_t` adhere to the `IsMereologicalLattice` consistency axioms—we confirm that the machine-level behavior is isomorphic to the formal mathematical theory. In this posture, the built-in types serve as the "Initial Models" for our concepts. If the compiler successfully verifies that a standard integer under bitwise conjunction satisfies our `IsMeetSemilattice` proof, it provides an automated, formal confirmation that our C++ concepts correctly capture the essential structural invariants found in the mathematical literature.

3 Implementation: The Functor Highway

The operational efficiency of `dedekind` is achieved through a technique we term *Synthetic Inference*. This level represents the "Skeletal Foundation" of the ontology, where machine-level instructions are unified with the abstract laws of Category Theory. By treating the C++ type system as a formal proof assistant, we ensure structural integrity via compile-time verification, or *Static Mereology*.

3.1 The Action-First Bootstrapping Strategy

In textbook Category Theory, a Monad is traditionally defined as a *Monoid in the category of Endofunctors*. However, the structural implementation of this definition in C++ faces significant language constraints:

1. **Partial Specialization Limitations**: C++ forbids the partial specialization of function templates (e.g., `fmap<F>`), preventing a centralized, generic definition for disparate species.

2. **ADL Resolution**: Function templates called via explicit name-lookup cannot trigger Argument-Dependent Lookup (ADL), making it difficult for the ontology to "discover" mappings for types defined in downstream modules.

To resolve this circular dependency, `dedekind` implements an *Action-First* bootstrapping strategy. By defining Monads and Comonads as *Extension Systems* (Kleisli Triples consisting of η/ε and $\gg= / \ll=$), we utilize operator overloading on concrete objects. This triggers ADL at the call site, allowing the Level 0 foundation to instantiate a derived `fmap` without prior knowledge of Level 1 species.

3.2 The Unified Highway Bridge

The `fmap` dispatcher serves as the implementation of this strategy. It automates the derivation of functorial mappings from the underlying monadic "Push" or comonadic "Pull."

```
export template <template <typename...> typename F, typename Arrow,
                 typename T = typename Arrow::Domain,
                 typename U = typename Arrow::Codomain>
  requires IsArrow<Arrow, T, U>
constexpr IsArrow<F<T>, F<U>> auto fmap(Arrow f) {
  if constexpr (std::same_as<F<T>, T>) {
    return f; // Identity Functor
  } else if constexpr (IsKleisliExtension<F, T, U>) {
    /** @theorem fmap(f) = m >>= (eta o f) */
    return arrow<F<T>, F<U>>([f](const F<T>& m) {
      return m >>= [f](const T& x) { return into<F, U>(f(x)); };
    });
  } else if constexpr (IsCoKleisliExtension<F, T, U>) {
    /** @theorem fmap(f) = w <<= (f o epsilon) */
    return arrow<F<T>, F<U>>([f](const F<T>& w) {
      return w <<= [f](const F<T>& ctx) { return f(extract<F, T>(ctx)); };
    });
  } else {
    static_assert(sizeof(T) == 0, "Ontology Error: Species lacks highway structure."
  }
}
```

Listing 5: The Unified Highway Bridge (fmap derivation)

3.3 Highway Notation: Directional Vectors

To facilitate a readable "Algebra of Programming," we map categorical data flows to directional operators, creating a "Highway Notation" that reflects the vector of computation across the ontology:

- `value » into<F>` : Represents η (Unit), lifting a species into a context.
- `box « extract<F>` : Represents ε (Counit), sampling a species from a context.
- `box »= f` : Represents *Bind*, the forward monadic composition (Left-to-Right).
- `box «= f` : Represents *Extend*, the contextual comonadic composition (Right-to-Left).

```
// Convert plain lambdas into tagged endomorphisms:
auto f = endo<int>([](int x) { return x + 1; });
auto g = endo<int>([](int x) { return x * 2; });
```

```
// Morphism composition: (h = g . f):
auto h = f >> g;

// Lifting into the Box and applying the composed morphism:
auto result = 5 >> into<Box> >> fmap<Box>(h);

CHECK(result == Box<int>{12});
```

Listing 6: Example of fused Kleisli "Highway" composition via the functor law.

```
// Define two monadic arrows (Species -> Boxed Species)
auto f = [](int x) { return pure(x + 1); };
auto g = [](int x) { return pure(x * 2); };

// Chain composition via Bind (>>=)
// Starting with a raw value, lifting it, then chaining the monadic arrows.
auto x = (5 >> into<Box> >>= f) >>= g;

// (5 + 1) * 2 = 12, wrapped in a Box
CHECK(x == Box<int>{12});

// Verification of the Monadic Associativity Law:
// (m >>= f) >>= g  is equivalent to  m >>= (x => f(x) >>= g)
auto lh = (5 >> into<Box> >>= f) >>= g;

auto rh = 5 >> into<Box> >>= [&](int x) { return f(x) >>= g; };

CHECK(lh == rh);
STATIC_CHECK(std::same_as<decltype(x), Box<int>>);
```

Listing 7: Chain Composition (Bind / »=)

This mapping ensures that the "Categorical Cement" holding the "Algebraic Bricks" together is both syntactically expressive and computationally invisible.

4 Implementation: The Compile-Time Logic Engine

The core of `dedekind` is a recursive template evaluator that treats C++ `concepts` as logical predicates. Unlike traditional template metaprogramming that relies on SFINAE, we utilize C++23's `requires` expressions to perform "structural inspection" of types.

4.1 The Compiler as a Deterministic Proof Assistant

In the `dedekind` framework, the C++ compiler acts as a decidability engine for our mathematical ontology. Rather than relying on traditional class hierarchies, we use C++23 Concepts to define the structural requirements of a species. A type is not explicitly inherited from a base; instead, it is verified to be compliant with a species' `concept` through its intrinsic properties. This allows for a structuralist approach where any type—primitive or user-defined—can participate in the mereology if it satisfies the necessary algebraic invariants. This verification is supported by Template Specialization to define axiomatic base cases and SFINAE to prune invalid overloads. We leverage the Curiously Recurring Template Pattern (CRTP) within our internal wrappers to provide a unified interface for disparate types without the overhead of dynamic dispatch. By conducting these checks in a `constexpr` context and utilizing Move Semantics for resource management, we ensure that the "Proof" of a mathematical construction is resolved entirely at compile-time, leaving only optimized machine code.

```
// Verifying that the intersection of disjoint sets is empty
using Evens = dedekind::set<int, [](auto x){ return x % 2 == 0; }>;
using Odds = dedekind::set<int, [](auto x){ return x % 2 != 0; }>;
using Intersection = dedekind::meet_t<Evens, Odds>;
// The compiler resolves this to the Empty Set species
static_assert(dedekind::is_empty_v);
static_assert(sizeof(Intersection) == 1); // No machine state required
```

Listing 8: Compile-time inference of set invariants

```
// In dedekind, a valid program is a constructive proof.
// If the 'Proof' (the type-transformation) fails, the 'Program' does not compile.
template
requires dedekind::is_archimedean_field_v
auto compute_limit(T sequence) {
return sequence.limit();
}
// Attempting to compute a limit on a Discrete Ring:
// The compiler acts as a proof-assistant and rejects the invalid mathematical path
// Error: constraints not satisfied for 'is_archimedean_field_v'
auto result = compute_limit(my_discrete_ring);
```

Listing 9: The Reverse Curry-Howard Isomorphism in action

4.2 Reifying the Set-Builder

To achieve the syntax

$$\text{Set}\{x \,\%\, \mathbb{R} \wedge (x + 5 < 10)\} \tag{2}$$

, we introduce a `set` template that accepts a domain and a predicate. The predicate is not a function pointer, but a *Stateless Constraint Type*.

```
import dedekind.numbers;
import dedekind.logic;
// Define a predicate: x is even and greater than 10
auto P = [](auto x) {
return (x % 2 == 0) && (x > 10);
};
// Materialize the set at compile-time
using EvenGreaterTen = dedekind::set<int, P>;
static_assert(dedekind::contains(12));
static_assert(!dedekind::contains(7));
```

Listing 10: Example of Set-Builder realization in Dedekind

4.3 The Role of C++23 Modules

The stratification mentioned in Section 2 is strictly enforced via the module system. By using `export module dedekind.ontology:mereology`, we ensure that the compiler can cache the "structural truths" of the system. This prevents the exponential blow-up of compile times typically associated with heavy template libraries, as the "laws" of the ontology are compiled once into binary module interfaces (BMIs)

5 Conclusion

This paper has explored the feasibility of embedding formal mereology within the C++23 type system through the `dedekind` library. By transitioning from the traditional object-oriented

paradigm of information hiding toward a model of structural transparency, we have illustrated how the compiler can be utilized as a rudimentary constraint satisfaction engine. This approach allows for the high-level representation of mathematical species, such as set-builder notation, without the typical runtime performance penalties associated with abstract modeling.

Our investigation suggests that while the Clang/LLVM toolchain is not a dedicated theorem prover, its capacity for aggressive optimization can be significantly enhanced when provided with formal semantic context. By hoisting mereological relations into the type signature, we enable the compiler to perform "structural pruning"—collapsing contradictions and optimizing intersections—resulting in machine code that maintains the efficiency of hand-tuned assembly.

The stratification of these formalisms into module partitions provides a scalable pathway for managing the "template tax," ensuring that the rigors of formal logic do not destabilize the build pipeline. By embedding the Dedekind cut and the construction of the Continuum into the build process, we have moved toward a nascent paradigm of *Axiomatic Systems Programming*: a development style where the build process acts as a physical guardrail for mathematical truth.

Future work will investigate extending this mereological framework to other domains of the computational sciences and high-performance computing, such as a direct integration with the upcoming linear algebra capabilities of the C++ standard library [11] while expressly facilitating computations not just using floating point numbers but more generally supporting simplified algebraic systems such as semimodules over rings ($\mathbb{B}, \mathbb{N}$) or extended number systems such as the complex numbers ($\mathbb{C}$), the dual numbers or quaternions.

References

[1] ISO/IEC JTC 1/SC 22. *ISO/IEC 14882:2024: Programming languages — C++*. Commonly referred to as C++23. International Organization for Standardization. Geneva, Switzerland, 2024. URL: `https://www.iso.org`.

[2] Bartosz Milewski. *Category Theory for Programmers*. An excellent bridge between C++ type systems and categorical structures. Blending Science, 2018.

[3] Stewart Shapiro. *Thinking about Mathematics: The Philosophy of Mathematics*. See Part IV for an accessible introduction to structuralism. Oxford University Press, 2000.

[4] The GHC Development Team. *The Glasgow Haskell Compiler User's Guide, Edition 2024*. GHC 2024 Language Edition. 2024. URL: `https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/control.html`.

[5] Ulf Norell. "Towards a practical programming language based on dependent type theory". PhD thesis. Chalmers University of Technology, 2007. URL: `http://www.cse.chalmers.se`.

[6] Thomas Mormann. "Structural Mereology and Statistics". In: *Logic and Logical Philosophy* 22.4 (2013), pp. 427–453.

[7] F William Lawvere. "An elementary theory of the category of sets (long version) with commentary". In: *Reprints in Theory and Applications of Categories* 11 (2005), pp. 1–35.

[8] Joachim Lambek and Philip J Scott. *Introduction to Higher-Order Categorical Logic*. Cambridge University Press, 1988.

[9] nLab authors. *structural set theory*. ncatlab.org/nlab/show/structural+set+theory. 2024. URL: `https://ncatlab.org`.

[10] Jean-Pierre Marquis. "The Structuralist Mathematical Style: Bourbaki as a Case Study". In: *The Architecture of Modern Mathematics*. Oxford University Press, 2022.

[11] Mark Hoemmen et al. *P1673R13: A free function linear algebra interface based on the BLAS*. Tech. rep. ISO/IEC JTC1/SC22/WG21, 2024. URL: `https://wg21.link`.